

Evaluating First-Attempt Zero-Shot GPT-4o Unit Test Generation on Java and Python Methods with Controlled Cyclomatic Complexity

Pham Vo Hung Nghi, Nguyen Tuan Vinh, Nguyen Hoang Gia Thai, Vuong Minh Quan, Nguyen Tuong Van
FPT University, Vietnam

Abstract—Writing unit tests manually remains time-consuming and error-prone, yet no prior empirical study has evaluated first-attempt zero-shot LLM test generation on a complexity-controlled, language-balanced sample with a pre-registered statistical analysis plan. We apply `gpt-4o-mini-2024-07-18` to 50 standalone functions (25 Java, 25 Python) filtered to cyclomatic complexity 5–15, generate one test suite per unit without any iterative repair, and measure branch coverage (JaCoCo and `coverage.py`) and mutation score (PIT for Java; Randoop 4.3.4 as a paired baseline). For RQ1, the one-sided sign test rejects the null that median branch coverage is at most 75% for both Java ($p = 1.49 \times 10^{-6}$, 23/24 units above threshold) and Python ($p = 4.28 \times 10^{-4}$, 19/22 units above threshold), with Java median branch coverage 95.5% and Python median 100%. For RQ2, GPT-generated tests outperform Randoop on mutation score across all 24 paired Java units (median difference +62.1 pp, 95% CI: 31.6–84.6 pp; Wilcoxon $p = 6.58 \times 10^{-5}$; Cliff’s $\delta = 0.79$, large effect). These results suggest that a single context-enriched prompt is sufficient to achieve high test quality on CC-moderate standalone functions, and that semantic understanding confers a decisive advantage over random test generation for functions with complex, domain-specific branching logic.

Index Terms—large language models, unit test generation, mutation score, empirical study

I. INTRODUCTION

Writing unit tests manually is a time-consuming and error-prone process, but complete test coverage remains essential for detecting logical errors before program release. For functions with a cycle complexity greater than 5, the range of forks becomes so broad that developers must meticulously list boundary states, boundary conditions, and exception paths—a task that is geometrically time-consuming compared to the amount of code being tested. The need for scalable automated test generation is therefore well-recognised in both industry and research.

Recent studies have examined large language models (LLMs) as a promising alternative to traditional search tools such as EvoSuite and Randoop. Schäfer et al. [1] have shown that `gpt-3.5-turbo` achieves an average of 52.8% branch coverage for JavaScript, TypeScript, and Python functions using the command line without training. Yuan et al. [2] have also shown that reloading a large language model with compiler feedback within a single revision cycle increases branch coverage to over 82% at the Java class level. Lops et al. [3] reported up to 79.8% branch coverage across three model configurations applied to a Java benchmark. Celik and

Mahmoud [4] found that LLM-generated tests outperform Randoop on mutation score when evaluated on isolated functions. These studies establish that LLMs can produce high-quality tests; however, direct comparisons between them are confounded by differences in generation protocol (single-shot versus iterative repair), level of analysis (method versus class), and the structural characteristics of the evaluated code.

No prior study has simultaneously (a) filtered subject functions to a controlled cyclomatic complexity band, (b) evaluated a single first-attempt response without iterative repair, and (c) compared against a classical automated baseline within a pre-registered statistical design. As a result, it remains unclear how code complexity affects first-attempt LLM test quality, and whether the observed improvements over random tools hold when complexity is held constant.

In this paper we address this gap by evaluating `gpt-4o-mini-2024-07-18` on 50 standalone functions (25 Java, 25 Python) with cyclomatic complexity strictly between 5 and 15, using a single context-enriched prompt and no iterative repair. We measure branch coverage and mutation score under a pre-registered analysis plan, and we compare the Java results directly against Randoop 4.3.4 using a paired design.

To summarise, this paper contributes:

- (1) The first pre-registered, complexity-controlled empirical evaluation of first-attempt zero-shot LLM test generation on a balanced Java/Python sample, measuring both branch coverage and mutation score.
- (2) Experimental results showing that `gpt-4o-mini-2024-07-18` achieves median branch coverage of 95.5% (Java) and 100% (Python) on CC 5 to 15 functions, with both populations rejecting the 75% null at $p < 0.001$.
- (3) A paired comparison against Randoop on Java mutation score demonstrating a 62.1 percentage-point median advantage for GPT-generated tests (Cliff’s $\delta = 0.79$, $p < 0.001$).
- (4) A public replication package including the frozen manifest, all API request/response logs, and the pre-registered analysis script at <https://github.com/phamvohungnghi/SWT301-SE1944-Group6>.

The rest of this paper is structured as follows. Section II reviews related work on LLM-based and search-based test generation. Section III describes our experimental design, dataset, model configuration, and statistical analysis plan.

Section IV reports the results for each research question. Section V interprets the findings and compares them with prior work. Section VI discusses threats to validity. Section VII concludes and outlines future work.

II. RELATED WORK

Using Large Language Models (LLMs) to automate unit test generation offers a compelling alternative to traditional random or search-based testing. We structure our review of recent progress in this field around two main areas: zero-shot LLM test generation and feedback-driven iterative refinement. We then highlight the gaps in these existing works and position our study accordingly.

A. LLM-based Zero-shot Unit Test Generation

Initial work in LLM-based testing focused on whether models could generate working unit tests immediately, without any iterative fixing. Guilherme and Vincenzi [5] took this approach with `gpt-3.5-turbo`, evaluating it on 33 Java projects. Under a 0.2 temperature and carefully formatted prompts, the model achieved a 64.6% build success rate, 90.2% average coverage, and a 70.5% mutation score. Schäfer et al. [1] followed up with TestPilot, a tool they used to benchmark `gpt-3.5-turbo`, `code-cushman-002`, and StarCoder across JavaScript, TypeScript, and Python. While they reported a median statement coverage of 70.2% and branch coverage of 52.8%, the resulting test suites often failed to compile due to syntax issues or broken external dependencies.

Looking beyond single languages, Celik and Mahmoud [4] developed the GEM framework to benchmark Codex, Sonnet, and Qwen on Java, Python, and C++ code. On the ULT(100) dataset, Codex hit 84.30% branch coverage and a 72.71% mutation score, though the authors noticed significant performance gaps when switching languages. In a different approach, Baskaran et al. [6] turned to multi-agent architectures, orchestrating OpenAI models with AutoGen to test a Python insurance app. This team-of-agents setup pushed branch coverage to 99% and mutation scores to 95.8%, though they only evaluated the framework on one specific software system.

However, zero-shot test generation struggles heavily with compilation and execution rates on the first try. Lops et al. [3] evaluated newer models like `gpt-4o-mini` and `gemini-1.5-pro` on the class-level Java benchmark CLASSES2TEST. Although the models reached up to 79.8% branch coverage and 89.2% mutation scores, the compilation success rate peaked at just 36.0%, showing a massive divide between test quality metrics and actual runnability. Crucially, these existing benchmarks do not control for the structural characteristics of the input code, such as its cyclomatic complexity. As a result, we still lack a clear understanding of how code complexity affects the build rates and quality of initial LLM outputs.

B. Feedback-Driven and Iterative Test Refinement

Recognizing that first-try tests often fail, researchers began adding compiler and execution feedback to let models debug

their own code. ChatTester, proposed by Yuan et al. [2], is one such tool; it feeds compilation errors back to the LLM to repair Java class tests. By letting the model learn from its mistakes, ChatTester boosted compilation rates by 34.3%, eventually reaching an 82% branch coverage and a 65% mutation score.

Along a similar line, Pan et al. [7] designed ASTER, which pairs GPT-4 with static analysis and context extraction to resolve errors in Java and Python tests. ASTER reached 78.0% line coverage and 77.2% branch coverage on Python benchmarks, winning approval in a developer study. Instead of fixing compiler errors, Straubinger et al. [8] used mutation testing results to drive a debugging loop, raising branch coverage from 55% to nearly 80% and mutation scores from 60% to 80%. Lu et al. [9] took this further by using DeepSeek-V3 to target survived mutants specifically, improving mutation scores by 16.04% for standalone methods and 8.11% for non-standalone methods.

Even though these feedback loops improve coverage and compilability, they come with high operational costs. Multiple round-trips to LLM APIs inflate token costs, increase execution delays, and require complex local runtimes and static analysis setups. More importantly, because these frameworks combine generation and healing, they hide the base model’s standalone zero-shot capabilities. We cannot easily see how source code complexity impacts the initial output quality when it is immediately masked by corrective iterations.

C. Research Positioning

We position our work to address two specific gaps. First, past evaluations of zero-shot generation do not isolate cyclomatic complexity; they use benchmarks with arbitrary complexity, making it hard to see if complex code is what breaks the LLM’s performance. We address this by testing `gpt-4o-mini-2024-07-18` on a balanced, complexity-controlled dataset of Java and Python methods (with cyclomatic complexity strictly between 5 and 15). Second, most current papers rely on simple descriptive statistics to compare LLMs with random testing baselines, lacking statistical rigor. We resolve this by applying non-parametric tests—specifically the one-sided sign test per language and the paired Wilcoxon signed-rank test—to confirm whether the improvements in branch coverage and mutation scores are statistically significant under a pre-registered analysis plan.

III. METHODOLOGY

This section describes our empirical study methodology, outlining the dataset preparation, the large language model setup, the prompt engineering design, the metrics measured, and the statistical analysis plan.

A. Dataset and Subject Units

We evaluate test generation on 50 standalone units: 25 Java methods and 25 Python functions. All units were selected from public open-source repositories and from a shared course project repository at <https://github.com/phamvohungnghi/SWT301-SE1944-Group6>.

Complexity filter. Cyclomatic complexity (CC) was measured using Lizard 1.23.0 on the source revision identified by the commit SHA recorded in the dataset manifest. Only units with $5 \leq CC \leq 15$ were eligible. This band captures functions with genuine branching logic—too complex for a trivial line-coverage sweep but small enough to be self-contained and runnable without external services.

Isolation requirement. Each unit must build and execute without network access, database connections, wall-clock dependencies, or uncontrolled global state. Two reviewers assessed isolation independently; any disagreement was resolved by the project lead.

Manifest and freeze. All eligible units were enumerated, assigned a `unit_id`, and sorted by the SHA-256 hash of the `unit_id` string. The first 25 Java and 25 Python entries were selected, yielding a deterministic, cherry-pick-resistant sample. The resulting manifest (`experiment/dataset-manifest.csv`) records, for each unit: `unit_id`, language, test framework, repository URL, commit SHA, license, relative path, start/end lines, function name, Lizard CC score, isolation status, SHA-256 source hash, and reviewer sign-offs. The manifest was frozen and committed before any API call was made.

Descriptive statistics. Cyclomatic complexity ranges from 5 to 15 for all 50 units by construction. The Java units span 13 distinct CC values (5 to 15); the Python units span the same range. All functions are standalone: they accept and return primitive or standard-library types without requiring test doubles for the production context.

B. LLM-based Test Generation Setup

For the automated test generation pipeline, we utilized the official OpenAI API using the `gpt-4o-mini-2024-07-18` model snapshot. To ensure the reproducibility of results and minimize bias, the parameters generating the results are tightly controlled:

- **Temperature:** Set to 0.0 to enforce deterministic code generation.
- **Top P:** The API requirements have been simplified to rely solely on temperature-based sampling.
- **Max Output Tokens:** The limit has been fixed at 2048 to avoid the possibility of the response being cut short.
- **System Prompt:** The system guided the model to operate like a software testing expert, adhering to the required framework, returning only executable source code, avoiding Markdown formatting blocks, and refraining from accessing external resources or modifying the production code.

At first (as documented in the revision log), we evaluated the process using `gpt-4o` via the GitHub Models platform. However, to address protocol discrepancies, collect accurate token consumption data via the `API usage` field, and obtain standard OpenAI response identifiers (`resp_*`), we switched to the official OpenAI API and reran all tests on 2026-07-16.

C. Prompt Engineering and Context Injection

A key challenge in first-attempt zero-shot unit test generation is ensuring the generated code compiles and is syntactically correct the first time. Our test evaluations show that without class structure and module context, LLMs often produce invalid class definitions in Java or make incorrect import assumptions in Python.

To overcome these issues, we created a prompt structure that includes context for each language:

1) Language-Specific Instructions:

- *Java Instructions:* The model was told to call methods through the `JavaAlgorithms` class and forbidden from creating separate classes named after the unique unit identifiers (e.g., `JAVA_001`).
- *Python Instructions:* The model was told to import the target function using the syntax `from python_functions import <function_name>` and forbidden from copying the production code into the test file.

2) Context Injection:

We included the entire target source file (or the first 200 lines if the file size exceeded 50 KB) as background context to provide the model with the exact imports, packages, and helper functions available.

3) Target Code:

The specific target method’s source code was appended at the end of the prompt.

D. Execution Cost and Token Analysis

To evaluate the performance and efficiency of our proposed framework, we conducted extensive experiments using the official API endpoint. A batch-processing run consisting of 60 successful requests was executed (10 requests during the pilot phase and 50 requests during the full run), consuming a total volume of 323,246 input tokens. The average payload per request was approximately 5,387.4 input tokens, indicating a highly dense contextual input/output interaction due to the injection of source code context. The total computational budget incurred during this empirical session was \$0.0789 USD, demonstrating the high cost-effectiveness of our methodology.

Table I summarises the token consumption and estimated cost using OpenAI’s pricing structure for the `gpt-4o-mini-2024-07-18` model.

TABLE I
TOKEN CONSUMPTION AND COST DETAILS

Phase	Input Tokens	Output Tokens	Total Tokens	Cost (USD)
Pilot (10 units)	53,831	9,205	63,036	\$0.0136
Full Run (50 units)	269,415	41,483	310,898	\$0.0653
Total (60 units)	323,246	50,688	373,934	\$0.0789

E. Metrics and Execution Tools

We measure three primary outcomes for each generated test suite: executability, branch coverage, and mutation score.

Executability. A generated test suite is classified as `EXECUTABLE` if it compiles successfully and all test methods

complete without a JVM crash under JUnit 5.11.4 for Java, or under pytest 9.1.1 for Python. Suites that fail to compile are classified as `COMPILE_ERROR`; suites that compile but abort at runtime are classified as `TEST_RUNTIME_ERROR`. Every generated response, including failures, is retained in the denominator when computing executability rates; no suite is silently dropped.

Branch Coverage. For Java, branch coverage is measured at method granularity using JaCoCo 0.8.13 instrumented through the Maven Surefire plugin. Only the target method’s branches are counted; helper code in the same class is excluded. For Python, branch coverage is measured per function using coverage.py 7.14.2 with the `--branch` flag and a per-function coverage run scope. Coverage is defined as $branch_coverage = \frac{covered_branches}{total_branches} \times 100\%$. Non-executable suites receive NA in the primary complete-case analysis and 0% in the worst-case sensitivity analysis.

Mutation Score. For Java, mutation score is measured using PIT 1.19.1 with its default mutator set. Timeout mutants are counted as survived in the primary analysis; invalid mutants are excluded from the denominator. Equivalent mutants are not removed by manual inspection after viewing results, to avoid inflating scores post-hoc. The formula is $mutation_score = \frac{killed}{generated+invalid} \times 100\%$. For Python, pytest-mutagen 1.3 is used. As a tool limitation, pytest-mutagen did not produce any valid mutants for any of the 25 Python functions; consequently, Python mutation scores are NA throughout and the Python mutation outcome is excluded from the confirmatory analysis.

Randoop Baseline. Randoop 4.3.4 is run for each of the 25 Java methods with `randomseed=20260621` and a time budget of 60 seconds per unit, using JDK 17 and the same production source revision as the GPT-generated tests. Randoop-generated suites are passed through the same JaCoCo and PIT pipeline without any manual repair. Raw command logs, generated test files, and per-unit coverage/mutation reports are archived in `evaluation/randoop/`.

F. Statistical Analysis Plan

All hypotheses are registered in `team-synthesis/proposal.md` before data collection. No threshold, test, or metric definition is altered after observing experimental results. The family-wise significance level is $\alpha = 0.05$. We report raw p -values, Holm-adjusted p -values, effect estimates, and 95% confidence intervals for all confirmatory tests.

RQ1 (Branch Coverage vs. 75% Target). We apply a one-sided exact sign test (right-tailed) separately for Java and Python, with H_0 : median branch coverage $\leq 75\%$. Observations exactly equal to 75% are excluded as ties. The two resulting p -values are adjusted for multiplicity using the Holm–Bonferroni procedure.

RQ2 (GPT-4o vs. Randoop, Java Only). We apply a one-sided paired Wilcoxon signed-rank test (right-tailed) on the per-unit differences $d_i = mutation_GPT_i - mutation_Randoop_i$, with H_0 : $median(d) \leq 0$. Zero differences are handled with the Wilcoxon zero-method. As a

sensitivity analysis, we apply a one-sided sign test excluding tied pairs. A deterministic bootstrap 95% confidence interval for the median difference is computed with seed 20260621 and 10,000 resamples.

RQ3 (CC vs. Test Quality). We compute one-sided Spearman rank correlations (left-tailed) between cyclomatic complexity and each of branch coverage and mutation score, separately for Java and Python. The four planned pairs yield three testable correlations after excluding Python mutation (which is `NOT_TESTABLE` due to the pytest-mutagen tool limitation). The three raw p -values are adjusted using Holm–Bonferroni correction. RQ3 is pre-registered as a secondary exploratory analysis; we do not claim causal relationships.

G. Protocol Amendments and Replication Package

Protocol amendments. Three deviations from the registered protocol were recorded in `notes.md` before any metric outcome was inspected. First, an initial pilot conducted on 2026-07-02 used the GitHub Models proxy (response IDs prefixed `mock-github-*`); this run was classified as a dry run and excluded from analysis. The full confirmatory run used the OpenAI API directly, producing authentic `resp_...` response IDs. Second, the target model was changed from `gpt-4o-2024-08-06` to `gpt-4o-mini-2024-07-18` for two reasons: (a) to reduce API cost to a total of USD 0.065 for 50 units, and (b) to increase output determinism by setting `temperature=0` (reduced from 0.2 in the proposal). Third, on 2026-07-16, the prompt was updated to supply the full production-file context—up to 200 lines of source—as the `MINIMAL_CONTEXT` field, resolving the class-name and import failures observed in the 2026-07-13 run. Only the 2026-07-16 outputs are used in the confirmatory analysis. All amendments and their rationales were recorded prior to the metric computation pipeline being executed.

Replication package. Our replication package includes: the frozen dataset manifest (`experiment/dataset-manifest.csv`) with SHA-256 source hashes and commit SHAs for all 50 units; prompt templates and their SHA-256 fingerprints; raw OpenAI request and response JSON for all 50 API calls; dependency lock files (`requirements-experiment.txt`) and Java build configuration; the pre-registered analysis script; raw JaCoCo, PIT, coverage.py, and Randoop output logs; and the full amendment log. The package is publicly available at: <https://github.com/phamvohungnghi/SWT301-SE1944-Group6>.

IV. RESULTS

This section reports the experimental results of the unit test suites generated by GPT-4o (zero-shot, first attempt) for 25 Python functions and 25 Java functions. We evaluated three aspects: executability, branch coverage, and mutation testing.

All results were obtained from `results/summary.csv` and `results/full_analysis.ipynb`. These files were generated by `evaluation/integrated_statistics.py` before any manual inspection or visualization.

A. Executability

Among the 25 generated Java test suites, 24 (96%) were classified as EXECUTABLE. The only non-executable test suite, JAVA_016_parseCsvLine, failed during compilation because the generated code referred to `unit_id` as a Java class name instead of `JavaAlgorithms`. This test suite was kept in the dataset, and its branch coverage and mutation score were recorded as NA in all main analyses.

Among the 25 generated Python test suites, 22 (88%) were classified as EXECUTABLE, while the remaining three (PY_013, PY_020, and PY_025) were classified as TEST_RUNTIME_ERROR due to syntax errors. More specifically, the generated test suites for PY_013_roman_to_int, PY_020_simple_html_tokenizer, and PY_025_is_match all produced the same error: `SyntaxError: unterminated string literal`.

We inspected the generated code and found the same cause in all three cases. The model generated very large mock data structures, such as long HTML token lists for `simple_html_tokenizer`, Roman numeral mapping tables for `roman_to_int`, and complex regular expression patterns for `is_match`. These outputs exceeded the API token limit, so the responses were truncated before completion. As a result, some string literals were never closed, leading to syntax errors. None of the generated test suites failed because of missing module imports or incorrect target function names. This indicates that the zero-shot system prompt provided enough context for GPT-4o to identify the correct source files.

B. RQ1 — Branch Coverage vs. 75% Target

Table II summarizes the branch coverage results and the sign-test analysis.

TABLE II
RQ1: BRANCH COVERAGE RESULTS (PRIMARY COMPLETE-CASE ANALYSIS)

Lang	n valid	Median	IQR	k above 75%	p_{raw}
Java	24/25	95.45%	[89.62, 100.00]	23/24	1.49×10^{-6}
Python	22/25	100.00%	[87.85, 100.00]	19/22	4.28×10^{-4}

For Java, the one-sided sign test produced $k = 23$ out of 24 non-tied observations above the 75% target ($p_{raw} = 1.49 \times 10^{-6}$, Holm-adjusted $p = 2.98 \times 10^{-6}$). For Python, 19 out of 22 non-tied observations were above the target ($p_{raw} = 4.28 \times 10^{-4}$, Holm-adjusted $p = 4.28 \times 10^{-4}$). The adjusted p -values are below $\alpha = 0.05$ for both Java and Python. Therefore, we reject H_0 for both programming languages.

Worst-case sensitivity analysis. We also performed a worst-case sensitivity analysis by assigning 0% branch coverage to every non-executable test suite. For Java, the sign test produced $p = 9.72 \times 10^{-6}$. For Python, the sign test produced $p = 7.32 \times 10^{-3}$. After assigning 0% branch coverage to non-executable test suites, both p -values remain below $\alpha = 0.05$.

A notable exception appeared in PY_006_validate_password (CC = 11). Although

the generated test suite was executable, it achieved only 8.33% branch coverage, covering only 1 of 12 branches. The execution log explains this result. The generated test suite contained 1,006 assertions placed one after another inside a single test function. Pytest ends the test function after the first failed assertion. The first failed assertion stopped the entire test function, so the remaining 1,005 assertions were never executed. This example shows a structural weakness of LLM-generated test suites. Large test functions with hundreds of assertions are easy to break because one failed assertion stops the entire test. As a result, the measured branch coverage drops to almost zero, even though most generated assertions are never evaluated.

C. RQ2 — GPT-4o vs. Randoop Mutation Score (Java)

Table III presents the paired comparison of mutation scores between GPT-4o and Randoop. Among the 25 Java functions, 24 provided paired mutation score data. JAVA_016_parseCsvLine did not produce a valid GPT-4o mutation score because its generated test suite could not compile. Therefore, this function was excluded from the paired analysis.

TABLE III
RQ2: PAIRED MUTATION SCORE — GPT-4o VS. RANDOOP (JAVA, $n = 24$)

Statistic	GPT-4o	Randoop
Median mutation score	86.61%	15.38%
IQR	[80.00, 95.21]	[6.25, 63.16]
Median difference (d_i)	+62.14 pp	
IQR of difference	[17.08, 86.16] pp	
Bootstrap 95% CI	[31.58, 84.62] pp	
Wilcoxon W	190.0	
p -value (one-sided)	6.58×10^{-5}	

We used a one-sided paired Wilcoxon signed-rank test to compare the mutation scores of GPT-4o and Randoop. The test produced $W = 190.0$ with $p = 6.58 \times 10^{-5}$, which is below $\alpha = 0.05$. Therefore, we reject H_0 . The median difference between the two approaches was +62.14 percentage points. The 95% bootstrap confidence interval ranged from 31.58 to 84.62 percentage points. GPT-4o outperformed Randoop in mutation testing on the Java benchmark. The median improvement was 62.14 percentage points.

We also performed a sensitivity analysis using a sign test. The analysis found 19 positive pairs, 0 negative pairs, and 5 ties, resulting in $p = 1.91 \times 10^{-6}$. The sign test produced the same conclusion as the Wilcoxon test. Both analyses indicate that GPT-4o consistently outperformed Randoop in mutation testing for the Java benchmark.

D. RQ3 — Cyclomatic Complexity vs. Test Quality

Table IV summarizes the results of the Spearman correlation analysis.

For Java, the Spearman correlation between cyclomatic complexity and branch coverage was $\rho = -0.517$, with a Holm-adjusted p -value of 0.0195. The adjusted p -value is

TABLE IV
RQ3: SPEARMAN CORRELATION — CC VS. TEST QUALITY
(HOLM-CORRECTED)

Language	Outcome	n	ρ	p_{raw}	p_{Holm}
Java	Branch Coverage	24	-0.517	0.0049	0.0195
Java	Mutation Score	24	+0.109	0.6946	1.0000
Python	Branch Coverage	22	-0.512	0.0074	0.0223
Python	Mutation Score	6	NaN	NaN	1.0000

below $\alpha = 0.05$, so H_0 is rejected. This result indicates a moderate negative correlation. In general, functions with higher cyclomatic complexity tended to achieve lower branch coverage.

For Python, the correlation between cyclomatic complexity and branch coverage was $\rho = -0.512$, with a Holm-adjusted p -value of 0.0223. This result is also statistically significant, so we reject H_0 . Similar to Java, more complex Python functions generally received lower branch coverage from the generated test suites.

For Java mutation scores, the correlation was $\rho = +0.109$, with a Holm-adjusted p -value of 1.0000. Therefore, we fail to reject H_0 . The results do not show a significant relationship between cyclomatic complexity and mutation score for the Java benchmark.

The correlation analysis could not be performed effectively for Python mutation scores because of two major limitations of the evaluation tool.

(a) **Manual mutant definition.** Unlike PIT for Java, which automatically injects mutants into bytecode, `pytest-mutagen 1.3` requires developers to define mutants manually using the `@mg.mutant_of` decorator in `test_mutations.py`. In this study, we created 29 mutants for the 25 Python functions to support mutation testing.

(b) **Base-failure abortion.** The second limitation comes from the execution policy of `pytest-mutagen`. The three functions with syntax errors automatically received NA because their generated test suites could not run. Another 16 executable functions also received NA. Although their test suites could run, they contained incorrect assertions that failed on the original (unmutated) program. To avoid false positive results, `pytest-mutagen` cancels mutation testing whenever the original test suite fails. As a result, only six functions completed mutation testing successfully: `bubble_sort`, `merge_intervals`, `calculate_tax`, `int_to_roman`, `parse_csv_line`, and `find_prime_factors`. However, none of these six test suites killed any of the defined mutants. Every mutant survived, so all six functions received a 0.0% mutation score. Because every valid mutation score was 0.0%, there was no variation in the data. As a result, Spearman’s correlation coefficient could not be computed, producing $\rho = NaN$.

Summary. Overall, the generated test suites achieved high branch coverage for both Java and Python. However, higher cyclomatic complexity generally reduced branch coverage in

both languages. For Java, GPT-4o also achieved much higher mutation scores than Randoop.

The Python results reveal another challenge. Many generated test suites achieved high branch coverage, but some generated assertions still failed on the correct implementation, especially for boundary cases. These failures prevented mutation testing for many functions. In addition, Python still lacks a mature and fully automated mutation testing framework comparable to PIT for Java. This limitation makes large-scale automatic evaluation more difficult.

V. DISCUSSION

A. What the Results Actually Show

RQ1 — Branch Coverage Against the 75% Mark

Both Java and Python cleared the pre-registered threshold with room to spare.

On the Java side, 23 out of 24 executable units finished above 75% branch coverage. The one exception was `dateDifference` (CC=8, coverage 66.7%)—a function whose correctness depends on locale-specific date handling, which the model did not probe. The sign test returned $p = 0.000001$ after Holm adjustment, so the null hypothesis is rejected with no ambiguity.

Python showed a different pattern in a good way: median coverage landed at an outright 100%. Nineteen of 22 executable units crossed the threshold (sign test $p = 0.0004$). The three that did not are worth examining individually. `validate_password` (CC=11) received tests that only walked the normal success path—the model generated inputs that passed every rule, leaving the rejection branches untouched. This is a familiar limitation noted in prior LLM testing work: validation functions with multi-clause conditions tend to get surface-level coverage. The other two low-coverage Python cases were not logic failures but tool failures, covered in the next paragraph.

Executability and What Failed

Java was almost clean: 24 of 25 functions ran. The one failure (`parseCsvLine`, CC=9) hit a compilation error unrelated to test logic—the generated class referenced an import that did not resolve.

Python was messier, though for a clear mechanical reason. Three functions—`roman_to_int`, `simple_html_tokenizer`, and `is_match`—came back with a `SyntaxError` pointing to an unterminated string literal. In each case, inspecting the raw response showed the file simply stopped in the middle of a string. The model ran into the `max_output_tokens` ceiling and the response was cut off. This is qualitatively different from the import/class-name failures that appeared in the earlier pilot run; those were fixed by including the full source file in the prompt, which is why the July 16 run was so much cleaner overall.

RQ2 — GPT vs Randoop on Mutation Score

This was the most decisive result of the study. Across all 24 matched Java pairs, the GPT-generated tests killed far more mutants than Randoop did. Median mutation score: 86.6%

for GPT, 14.8% for Randoop. The per-unit difference had a median of 62.1 percentage points (95% CI: 31.6% to 84.6%). Wilcoxon one-sided $p = 6.6 \times 10^{-5}$, Cliff’s $\delta = 0.79$ —a large effect by any standard classification.

Not a single unit went the other way. Randoop never outscored GPT on mutation; five units tied.

The functions where Randoop did worst tell the story clearly. On `isValidEmail`, Randoop achieved 4.8% mutation score because random string generation almost never produces a syntactically valid email address—the few that randomly satisfy the regex are not diverse enough to kill boundary mutants. GPT scored 100% on the same function. The same dynamic appeared on `evaluateRPN` (Randoop: 0.0%, GPT: 100%) and `jsonPathLookup` (Randoop: 0.0%, GPT: 87.5%). Random enumeration cannot construct domain-meaningful inputs without understanding the function’s contract.

Java Mutation Score as a Standalone Number

For reference, the median PIT score across all 24 Java units was 86.6% (IQR: 80.0%–95.5%, mean 85.7%). The lowest individual score was `checkSudokuBoard` at 54.6% (CC=13). That function has deeply nested row/column/box validation loops; the generated tests covered branches but did not build enough distinct board configurations to distinguish all the boundary mutants from one another.

B. How These Numbers Sit Against Earlier Work

The results are broadly in line with the better end of what prior studies have reported, but comparing them directly requires care because the study designs differ on several dimensions.

Schäfer et al. [1] measured a median branch coverage of 52.8% on JavaScript, TypeScript, and Python functions. Their pipeline allowed up to five repair iterations, so the output is not a raw first-attempt number. More importantly, they evaluated functions of all complexity levels without filtering—which means simple functions pull the median up while complex ones drag it down. Our CC 5–15 filter removes both extremes, and the full-context prompt eliminates most import failures, which their paper identifies as a main driver of test failures.

Lops et al. [3] reported 79.8% branch coverage. That figure is the highest result across three different models and multiple prompt configurations applied to Java classes rather than individual methods. A class-level analysis will naturally produce different numbers than a method-level one, and taking the maximum over a model grid is not comparable to a single first-attempt median.

Guilherme and Vincenzi [5] got a mean of 90.2% on 33 Java programs using `gpt-3.5-turbo`. Their sample had no complexity filter, so it almost certainly included many trivially easy functions (CC < 5) where any test will hit 100% coverage. That inflates the mean. The pattern we see in our own data is consistent with this: every unit in our set with the lowest CC values achieved 100% coverage without difficulty.

Yuan et al. [2] reached above 82% branch coverage and above 65% mutation score on Java classes through multi-turn

self-repair. The repair loop is doing real work in their pipeline. The fact that our first-attempt Java mutation median (86.6%) sits above their repaired mutation result is notable—it suggests that supplying the right context upfront can compensate for the lack of iterative correction, at least within the CC 5–15 range.

On the Randoop comparison specifically, Celik and Mahmoud [4] also found that LLM-generated tests beat random-based tools on mutation score for isolated functions. Our effect ($\delta = 0.79$) is larger than what they report. Part of that gap is likely attributable to the CC filter: by removing very simple functions where Randoop does fine through random enumeration, we are measuring performance on exactly the kind of code where semantic understanding matters most.

Three factors account for most of the differences between our results and earlier work: the level of analysis (individual method versus class or file), the generation protocol (one shot versus iterative repair), and whether complexity was controlled. No paper in our literature review did all three at once.

C. What This Means in Practice

For Development Teams

The practical takeaway is fairly direct. A single call to `gpt-4o-mini-2024-07-18` with a prompt that includes the full source file can routinely produce tests hitting above 95% branch coverage (Java) or 100% (Python) for standalone functions with CC between 5 and 15—at a cost of roughly USD0.07 for 50 functions. Compared to Randoop, GPT-generated tests found far more mutants, with a 62-percentage-point median advantage.

That said, two situations deserve extra human attention. First, validation-heavy functions—the kind that enforce complex multi-clause rules like password policies or format checks—tended to get only happy-path coverage. A reviewer should check these specifically. Second, if a test file comes back with a `SyntaxError` at the end, the likely cause is a truncated response rather than a logic error; bumping the token limit fixes it without needing to redesign the prompt.

For People Building Test Generation Tools

The main failure mode we observed has nothing to do with the model’s reasoning—it is a pure engineering issue. When the output gets cut off mid-string, the file is dead on arrival. A simple length check before handing the file to the compiler, with an automatic retry at a higher token limit (say, 4,096 tokens), would likely eliminate nearly all the non-executable cases we saw.

For Researchers Running Similar Studies

The protocol used here—frozen dataset manifest with SHA-256 hashes, prompt fingerprinting, pre-registered hypotheses, and all raw API logs committed to a public repository—took no extraordinary effort and cost almost nothing to run. The large RQ2 effect ($\delta = 0.79$) appeared without any fine-tuning, model customisation, or repair loop, which suggests that a well-scoped complexity filter does a lot of the work that researchers usually attribute to elaborate prompting strategies.

Extending this beyond CC 15 is a natural next step, as is running the same protocol under a different LLM to see how much of the result is model-specific versus prompt-structure-specific.

VI. THREATS TO VALIDITY

A. Internal Validity

Researcher degrees of freedom. The research questions, hypotheses, threshold (75%), statistical tests, and multiplicity-correction scheme (Holm) were registered in the proposal prior to data collection. The analysis script was finalised before the full run began. Three protocol deviations were recorded as formal amendments in `notes.md` before any metric was inspected, preventing post-hoc hypothesis adjustment.

Model non-determinism. We pin the exact model snapshot (`gpt-4o-mini-2024-07-18`) and set `temperature=0` to minimise output variance. We do not claim complete determinism because the provider does not guarantee it for parallel decoding; however, a single response per unit was collected and stored verbatim, making the study a single-point observation rather than a probabilistic claim.

Infrastructure retries. Network-level retries were permitted only for transport failures (HTTP 5xx before content delivery). After the API returned any content, no re-call was issued. This preserves the first-attempt intent of the study.

B. Construct Validity

Branch coverage as quality proxy. Branch coverage does not capture semantic correctness of test oracles. We mitigate this by using mutation score (PIT) as a co-primary metric for Java, which directly assesses the ability of tests to detect injected logical faults.

Python mutation score unavailable. `pytest-mutagen 1.3` requires explicit mutant decorators rather than automatic injection; consequently, zero mutants were generated for 20 of 22 executable Python units, and the remaining two aborted because the base test suite failed on the unmutated code. Python mutation score is therefore excluded from confirmatory analysis. This limits the comparability of quality evidence between the two languages.

Non-executable tests. One Java unit (`parseCsvLine`, CC=9) and three Python units produced non-executable tests. All four are retained in the denominator; our primary analysis uses complete-case (executable units only), and a worst-case sensitivity analysis assigns 0% to non-executable units. Both analyses agree in their rejection of H_0 for RQ1.

C. External Validity

Small and filtered sample. Our sample consists of 50 standalone functions with $5 \leq CC \leq 15$, drawn from a single frozen repository. Results should not be generalised beyond this complexity range or to class-level, file-level, or framework-dependent code. Effect sizes and 95% confidence intervals are reported throughout to aid readers in assessing practical significance.

Single model snapshot. All results are conditioned on `gpt-4o-mini-2024-07-18`. Performance of other models or future snapshots of the same model may differ. RQ3 (CC vs. quality correlation) is marked exploratory to caution against over-generalising the complexity-coverage relationship.

Randoop fairness. GPT-4o-mini and Randoop operate under identical conditions: same Java source revision, same JDK 17, same PIT toolchain, and a fixed Randoop time budget of 60 seconds per unit with a pinned random seed (20260621). The time budget was chosen to reflect a realistic CI-integration scenario; longer budgets may narrow the observed gap.

Tool and language heterogeneity. Java coverage is measured by JaCoCo and Python coverage by `coverage.py`; the two tools may differ in their branch-counting granularity for equivalent control-flow constructs. Confirmatory RQ1 tests are conducted separately for each language; pooled results are used only for descriptive purposes.

VII. CONCLUSION

Both Java and Python cleared the pre-registered threshold with room to spare. On the Java side, 23 out of 24 executable units finished above 75% branch coverage. The sign test returned $p = 0.000001$ after Holm adjustment, so the null hypothesis is rejected with no ambiguity. Python showed a median coverage of 100%, with 19 of 22 executable units crossing the threshold.

The head-to-head with Randoop was the most decisive result of the study. Across all 24 matched Java pairs, the GPT-generated tests killed far more mutants than Randoop did: median mutation score of 86.6% for GPT versus 14.8% for Randoop (median difference +62.1 percentage points, Wilcoxon $p = 6.6 \times 10^{-5}$, Cliff's $\delta = 0.79$). Not a single unit went the other way. Randoop struggled on functions requiring domain-specific valid inputs (such as `isValidEmail`, `evaluateRPN`, and `jsonPathLookup`), whereas GPT's semantic understanding allowed it to construct meaningful inputs effortlessly.

Our results are broadly consistent with prior literature but benefit from greater control. Unlike TestPilot [1] or AgoneTest [3], which evaluated arbitrary code without filtering complexity or allowed multi-turn repair, our study shows that supplying full production-file context upfront in a single zero-shot prompt achieves superior first-attempt quality within the CC 5 to 15 range.

For development teams, a single call to `gpt-4o-mini-2024-07-18` with full source context produces tests hitting above 95% branch coverage at a cost of under \$0.08 for 50 functions. Tool builders should note that the primary failure mode (string truncation) can be eliminated by implementing an automatic retry at 4,096 max tokens when syntax errors are detected.

Our complete replication package, including dataset manifests, raw API request/response logs, and pre-registered analysis scripts, is publicly available at <https://github.com/phamvohungnghi/SWT301-SE1944-Group6>.

REFERENCES

- [1] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering (TSE)*, vol. 50, no. 1, pp. 85–105, 2024. [Online]. Available: <https://doi.org/10.1109/TSE.2023.3334955>
- [2] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, “No more manual tests? evaluating and improving chatgpt for unit test generation,” *Proceedings of the ACM on Software Engineering (FSE)*, 2024. [Online]. Available: <https://doi.org/10.1145/3660783>
- [3] A. Lops, F. Narducci, A. Ragone, M. Trizio, and C. Bartolini, “Llms for automated unit test generation and assessment in java: The agonetest framework,” *arXiv preprint arXiv:2502.06556*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.06556>
- [4] A. Celik and Q. H. Mahmoud, “Gem: A framework for strengthening llm-generated unit tests using mutation feedback,” in *Proceedings of the Ibero-American Conference on Software Engineering (CIBSE)*, 2026. [Online]. Available: <https://sol.sbc.org.br/index.php/cibse/article/view/42441>
- [5] V. H. Guilherme and A. M. R. Vincenzi, “An initial investigation of chatgpt unit test generation capability,” in *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing (SAST)*, 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3624032.3624035>
- [6] S. Baskaran, P. Mishara, and R. Agarwal, “Llm driven unit test case generation using agentic ai,” *Journal of Ubiquitous Computing and Communication Technologies (JUCCT)*, 2025. [Online]. Available: <https://irojournals.com/jucct/article/view/1818>
- [7] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha, “Aster: Natural and multi-language unit test generation with llms,” *arXiv preprint arXiv:2409.03093*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.03093>
- [8] P. Straubinger, M. Kreis, S. Lukasczyk, and G. Fraser, “Mutation testing via iterative large language model-driven scientific debugging,” in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.08182>
- [9] Z. Lu, P. Zhang, Y. Nie, Y. Yang, Y. Tang, C. Y. Chong, and Y. Zhou, “Beyond coverage: Automatic test suite augmentation for enhanced effectiveness using large language models,” *Proceedings of the ACM on Programming Languages (PACMPL)*, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3798251>